

Academic Library Management System: Final Report

Safal Shrestha
shre4420@vandals.uidaho.edu
University of Idaho
Moscow, Idaho, USA

S Q Abdullah Al Mamun
mamu2549@vandals.uidaho.edu
University of Idaho
Moscow, Idaho, USA

Swikriti Shrestha
shre9317@vandals.uidaho.edu
University of Idaho
Moscow, Idaho, USA

Abstract

This project implements an Academic Library Management System using a graph-oriented relational design. The system models authors, institutions, publications, and venues as graph nodes. It also models relationships such as authorship, affiliation, citation, publication venue, and co-authorship as graph edges. The final prototype uses SQLite for the database, Flask for the backend, and HTML, CSS, JavaScript, and Cytoscape.js for the frontend and graph visualization. The system supports a complete working prototype with dashboard views, search tools, CRUD operations, graph analytics, export features, and interactive visualizations. It also fully implements the three required queries: co-author network visualization, h-index filtering, and Q1 journal influence network discovery. The final version shows that a graph-based model is useful for academic library data because it helps users explore collaboration, citation, and venue influence patterns more clearly than normal record-based tables.

Keywords

graph database, academic library, SQLite, Flask, Cytoscape.js, co-author network, h-index, Q1 journal, visualization

1 Introduction

Academic library data is not only a list of records. It also contains many important relationships. Authors write publications, publications cite other publications, authors work at institutions, and papers are published in journals or conferences. A normal library system can store this information, but it may not clearly show how everything is connected.

The goal of this project was to build a graph-oriented Academic Library Management System. Instead of treating authors, publications, institutions, and venues as separate records only, our system treats them as connected graph nodes. The relationships between them are stored as graph edges. This makes it easier to answer relationship-based questions, such as which authors collaborate with each other, which authors have high citation impact, and which researchers are connected to Q1 journals.

The final system is a complete working prototype. It includes a database schema, seeded academic dataset, backend API, browser frontend, graph visualizations, CRUD tools, search tools, and export features. The final dataset includes 18 authors, 6 institutions, 10 venues, 32 publications, and a larger set of graph relationships. The prototype also supports three named graph contexts: Collaboration Graph, Citation Graph, and Venue Influence Graph.

This report explains the finalized architecture, database design, working prototype, required queries, visualization features, evaluation, limitations, and future improvements.

2 Finalized Architecture

The final architecture has three main layers: the database layer, the backend layer, and the frontend layer.

The database layer uses SQLite. SQLite stores the graph-oriented relational schema. All graph entities are first stored in the shared Nodes table. Type-specific information is stored in subtype tables such as Authors, Institutions, Publications, and Venues. All relationships are stored in the shared Edges table. Multi-graph membership is supported through Graphs, NodeGraphs, and EdgeGraphs.

The backend layer uses Python and Flask. Flask serves the frontend files and provides REST API routes. The backend initializes the database, seeds demo data, validates user input, handles CRUD operations, computes graph queries, and returns JSON responses to the frontend.

The frontend layer uses HTML, CSS, and vanilla JavaScript. The user interface includes a dashboard, researcher exploration page, collections page, analytics page, management tools, and search tools. Cytoscape.js is used to render graph visualizations directly in the browser.

3 Database Design

The database design follows the project requirement that all nodes must be stored in a single Nodes table and all edges must be stored in a single Edges table.

The Nodes table stores the main identity of each graph object. Important fields include NodeID, NodeType, DisplayLabel, and AttributesJson. The supported node types are Author, Institution, Publication, and Venue.

The subtype tables store details that only belong to one entity type. The main subtype tables are:

- Authors(NodeID, FullName, ResearchArea, Email)
- Institutions(NodeID, Name, Country)
- Publications(NodeID, Title, PublicationYear, DOI)
- Venues(NodeID, Name, VenueKind, Quartile, ImpactScore)

The Edges table stores the relationships between nodes. Important fields include EdgeID, SourceNodeID, TargetNodeID, EdgeType, EdgeYear, Weight, and MetadataJson. The implemented edge types are:

- AUTHORED
- AFFILIATED_WITH
- PUBLISHED_IN
- CITES
- CO_AUTHOR

This structure keeps the database flexible. New node types and edge types can be added later without redesigning the whole system.

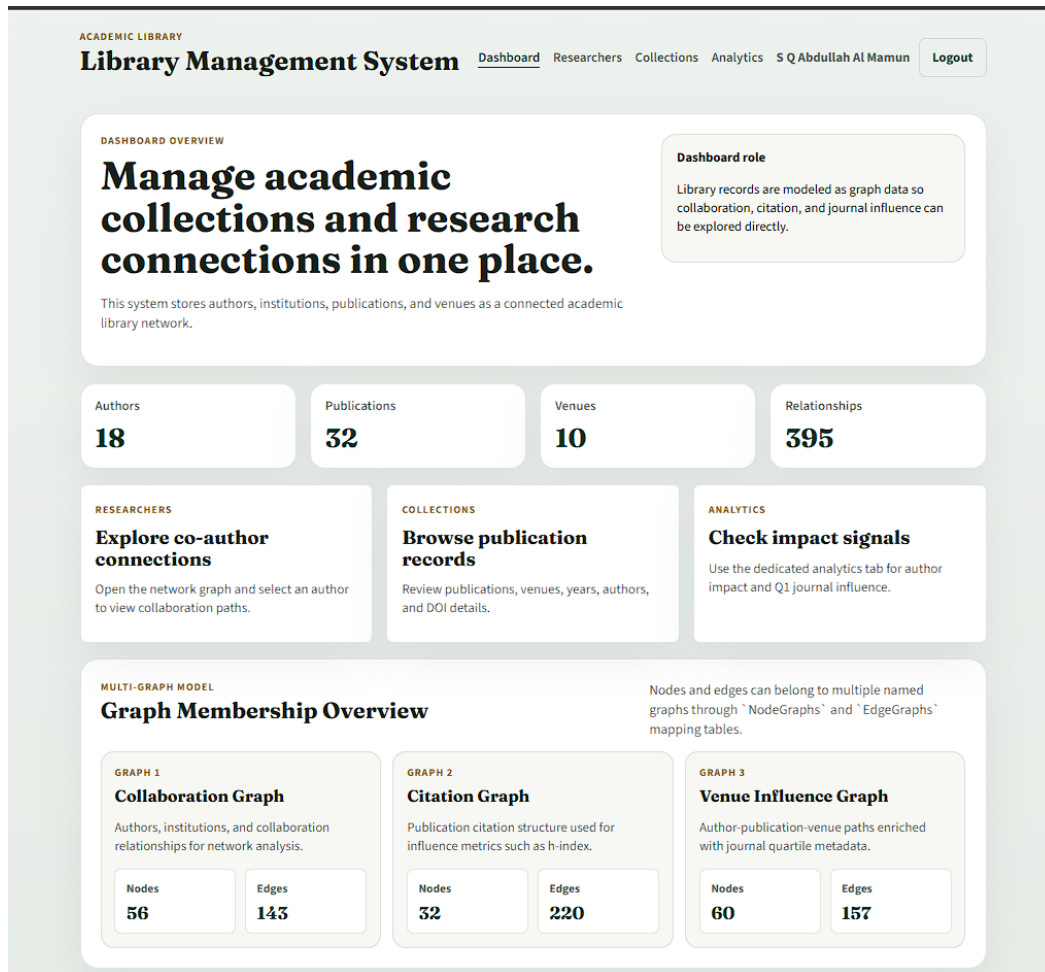


Figure 1: Dashboard page showing total authors, publications, venues, relationships, graph membership, and main navigation cards.

4 Multi-Graph Support

The system supports multiple logical graph views. This is important because the same data can be used in different ways. For example, an author may belong to the Collaboration Graph and also appear in the Venue Influence Graph.

The Graphs table stores the graph names. The final system uses three graph contexts:

- Collaboration Graph
- Citation Graph
- Venue Influence Graph

The NodeGraphs table maps nodes to graph contexts. The EdgeGraphs table maps edges to graph contexts. This allows nodes and edges to appear in more than one graph view. For example, an author node may appear in both collaboration analysis and Q1 journal influence analysis.

5 Complete Working Prototype

The final version is not only a database schema. It is a full working prototype with a browser-based interface.

The dashboard gives a quick summary of the dataset. It shows counts for authors, publications, venues, and relationships. It also gives access to the main sections of the application.

The Researchers page allows users to explore authors and co-author networks. Users can select an author and view the selected author profile, direct collaborators, publication history, and graph visualization.

The Collections page lets users browse publications, authors, institutions, and venues. It includes search and filter tools so users can find records by title, author, year, country, quartile, and venue type.

The Analytics page includes the required graph queries and additional analytics. It displays the h-index filter, Q1 influence network, influential author ranking, institution collaboration ranking, and shortest collaboration path.

The system also includes CRUD operations for authors, institutions, venues, and publications. Input validation was added for invalid email formats, duplicate DOI values, duplicate entity names, and unsafe deletion cases.

6 Implementation Details

The final implementation was built to make the system work as a complete web application instead of only a database design. The project is organized around the database files, backend application files, frontend pages, JavaScript logic, and test files. This made the system easier to develop and easier to demonstrate during the final presentation.

The backend application is written in Python using Flask. Flask was used because it is lightweight and simple to connect with SQLite. The backend is responsible for creating the database if it does not already exist, loading the seeded dataset, running SQL queries, and returning results as JSON. The frontend does not directly access the database. Instead, it sends requests to Flask API routes. This keeps the database logic in one place and makes the frontend easier to maintain.

The database initialization process is also important. When the application starts, it checks whether the SQLite database exists and whether the required tables are available. If the database is missing or empty, the system creates the schema and inserts the seed data. This makes the prototype easier to run on another computer because the user does not have to manually create every table.

The frontend is built with plain HTML, CSS, and JavaScript. We used this approach because it keeps the project simple and understandable for a class prototype. The JavaScript files call backend API routes using `fetch()` and then update the page with the returned data. For graph views, the frontend converts JSON nodes and edges into Cytoscape.js elements. Cytoscape.js then draws the graph in the browser.

The final prototype also includes export features. Users can export tabular results as CSV files and export graph views as PNG images. This is useful because the system is not only for viewing data but also for saving results for reports or presentations.

7 Frontend User Flow

The user flow begins at the login and signup screen. For the final prototype, this login system is used mainly for demonstration. After logging in, the user reaches the dashboard. The dashboard gives a high-level view of the academic library graph, including the number of authors, publications, venues, and relationships.

From the dashboard, the user can move to the Researchers page. This page is focused on author exploration. A user can choose an author and view the co-author network. The page also shows the selected author's publication history and collaborator information. This page is important because it demonstrates the first required query in a visual way.

The Collections page is used to browse academic records. It includes publication information, venue information, institution information, and search tools. This page helps show that the system is also a management system, not only a graph visualization tool. Users can search by different types of data and inspect records in a more traditional catalog format.

The Analytics page is used for higher-level graph analysis. It includes the h-index filter and the Q1 journal influence network. It also includes additional features beyond the minimum requirements, such as shortest collaboration path and ranking-style analytics. This page shows how the same node and edge structure can support multiple graph questions.

The Management tools support create, edit, and delete operations. This makes the prototype more realistic because library data usually changes over time. New authors, publications, venues, and institutions can be added to the system. Existing records can also be edited or removed if the deletion is safe.

8 Validation and Error Handling

Validation was added to make the final prototype more reliable. Without validation, users could enter incorrect or duplicate data, which would make the graph results less trustworthy.

For author records, the system checks that email values use a valid format. This prevents obviously incorrect contact data from being stored. For publication records, the system checks duplicate DOI values. This is important because DOI values are supposed to identify publications uniquely. If duplicate DOI values are allowed, the same publication could appear more than once in the graph.

The system also checks duplicate names where appropriate. For example, it helps prevent repeated venue or institution records. This keeps the dataset cleaner and makes search results easier to understand.

Unsafe deletion is another important validation case. For example, an institution should not be deleted if authors are still affiliated with that institution. If the system allowed this, it could create broken relationships in the graph. The validation rules help protect the graph structure and keep node-edge relationships consistent.

The final prototype is still not a production system, but these validation steps make it stronger than a basic demo. They show that the system was designed with data quality and usability in mind.

9 Required Queries

9.1 Query 1: Co-Author Network

The first required query retrieves the co-author network for a selected author. In this system, co-author relationships are stored as `CO_AUTHOR` edges. These edges are derived from shared publications. The edge weight represents how many publications two authors share.

When the user selects an author, the backend finds all direct `CO_AUTHOR` edges connected to that author. The response includes the selected author, collaborator nodes, edge weights, shared-publication summaries, publication history, and graph-ready JSON data.

The frontend uses Cytoscape.js to visualize the result. The selected author is shown as the center of the network. Connected co-authors appear around the selected author. Edge labels show shared publication counts.

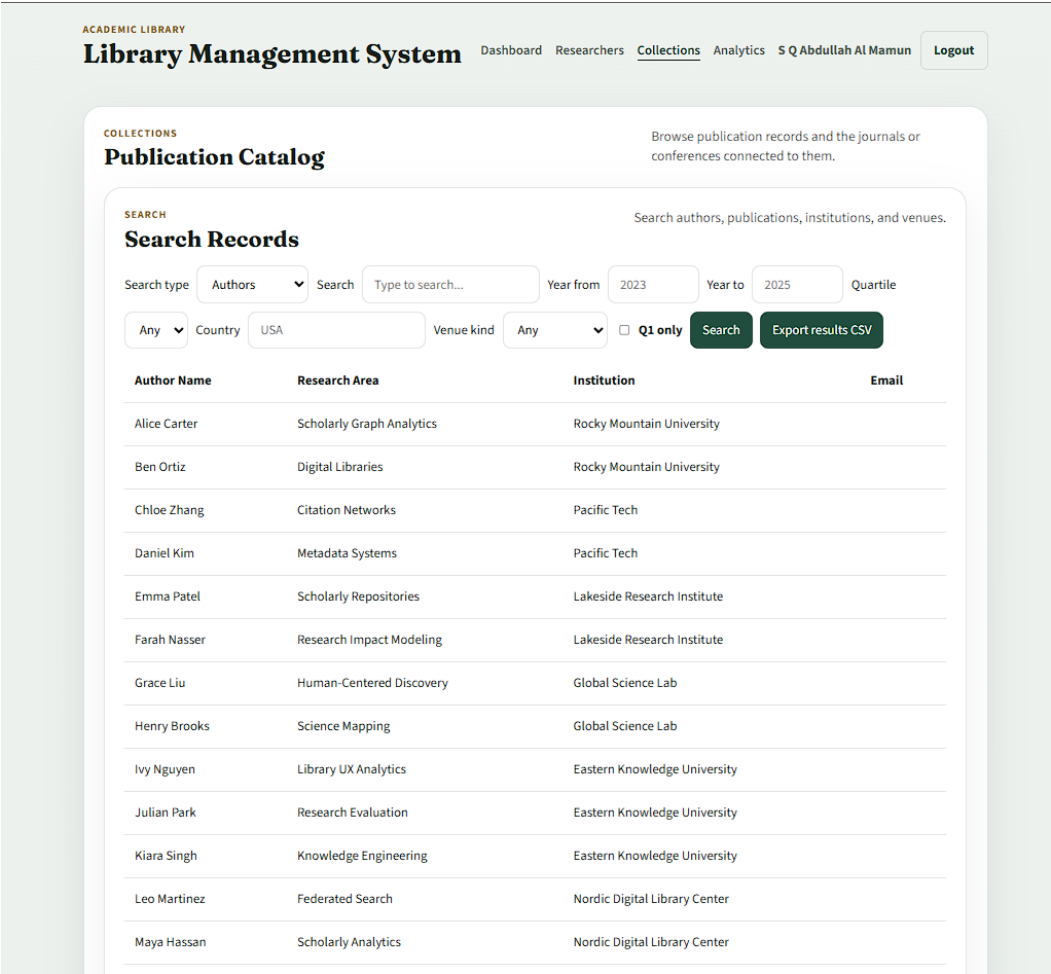


Figure 2: Collections page showing publication records, search records, institution authors, and venue information.

9.2 Query 2: H-Index Filter

The second required query finds authors with h-index greater than or equal to 5. In this project, h-index is calculated using incoming CITES edges. A publication receives a citation when another publication has a CITES edge pointing to it.

The backend groups publications by author. Then it counts the incoming citations for each publication. After sorting citation counts, the backend calculates h-index using the standard definition: an author has h-index *h* if at least *h* publications have at least *h* citations each.

The result includes author name, h-index value, publication venues, and top cited publications. This query connects citation graph data to author impact analysis.

9.3 Query 3: Q1 Journal Influence Network

The third required query identifies authors connected to authors who publish in Q1 journals. In this system, a Q1 journal is defined by the Quartile field in the Venues table, where Quartile = 'Q1'.

The backend first finds authors who have published in Q1 venues. Then it expands the result through CO_AUTHOR edges to find authors directly connected to them. The query returns qualifying authors, Q1-linked authors, and the graph elements needed for visualization.

The frontend displays the Q1 influence network as an interactive graph. This helps users see how top-tier journal influence may spread through collaboration networks.

10 Visualization Demonstration

Visualization is one of the most important parts of the final prototype. The system uses Cytoscape.js to display graph query results in the browser. This satisfies the project requirement for graph visualization and also makes the system easier to understand during a live demo.

The co-author network visualization shows the selected author, direct co-authors, and collaboration strength. The Q1 influence visualization shows authors who published in Q1 venues and authors connected to them through collaboration.

The frontend also supports interaction features. Users can click nodes to inspect details, highlight graph neighborhoods, resize the

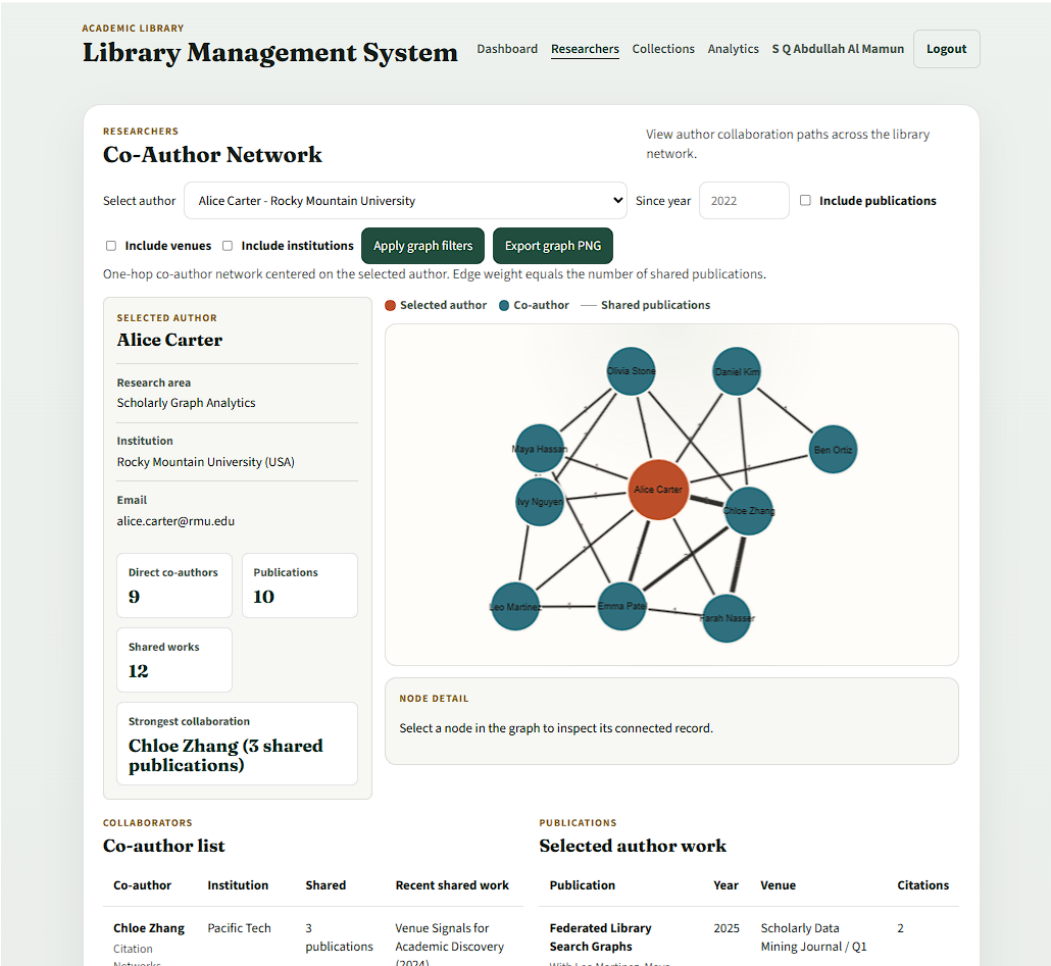


Figure 3: Researchers page showing the selected author, one-hop co-author network, collaborator list, and selected author publications.

graph, fit the graph to the page, and export graph images as PNG files. Some views also allow users to include publication, venue, and institution nodes in the visualization.

11 Testing and Verification

Testing was used to check whether the final system behaves correctly. The project includes tests for both graph logic and backend API behavior. These tests are important because the required queries depend on several connected tables. A small error in an edge query or join could produce incorrect graph results.

The tests check that the seeded dataset loads correctly. This includes checking expected counts for authors, publications, venues, and relationships. The tests also check graph membership so that nodes and edges appear in the correct graph contexts.

The required query tests check the co-author network, h-index filter, and Q1 influence network. For the co-author query, the tests confirm that the selected author returns the expected collaborators and graph edges. For the h-index query, the tests confirm that citation counts are interpreted correctly and that authors meeting

the threshold are returned. For the Q1 influence query, the tests confirm that Q1 venue metadata is connected correctly to author collaboration edges.

Additional tests check filtered search behavior, shortest collaboration path, influence ranking, and CRUD validation. This gives more confidence that the final application is working beyond only the required queries.

Testing also helped during development because it made problems easier to find. If a change broke one of the required queries, the test results showed the issue quickly. This made the final prototype more stable for presentation.

12 Evaluation

The final system was evaluated based on functionality, correctness, and usability.

From a functionality point of view, the system meets the main project requirements. It stores all graph nodes in one Nodes table, stores all graph relationships in one Edges table, supports

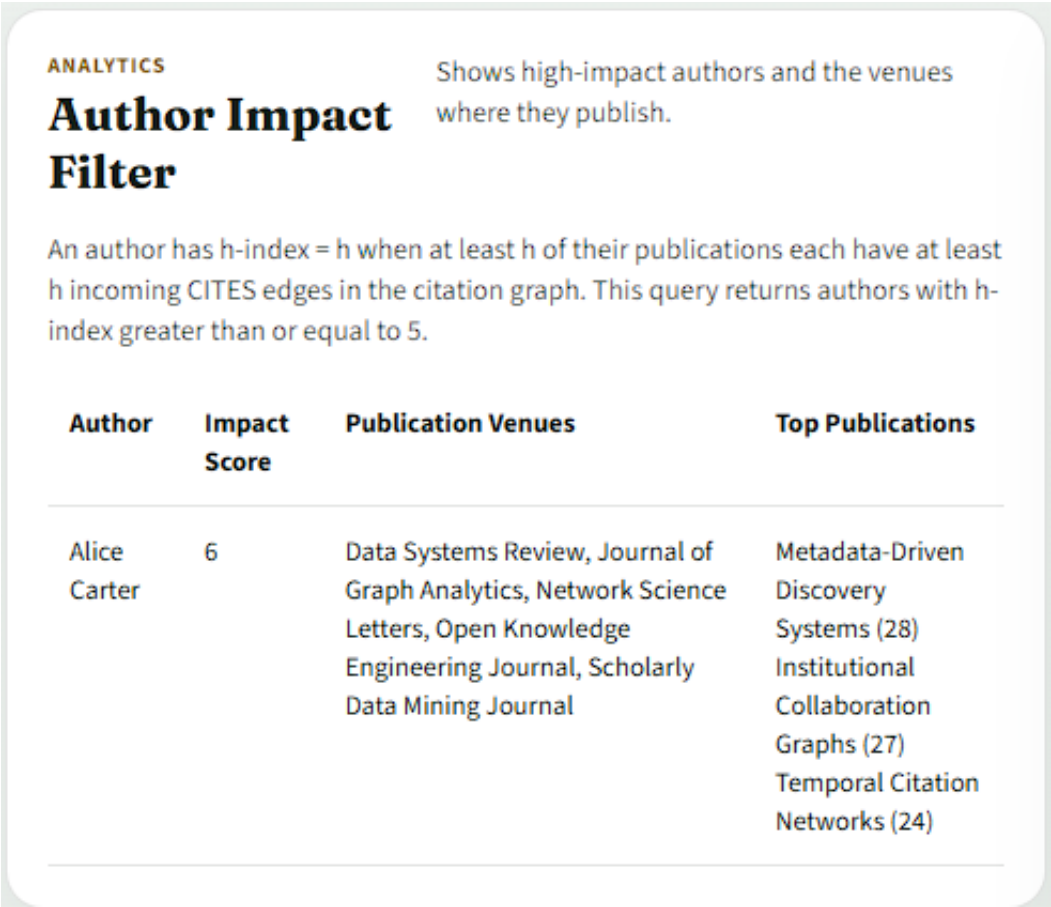


Figure 4: Analytics or collections view showing venue records, journal quartiles, publication counts, and impact scores.

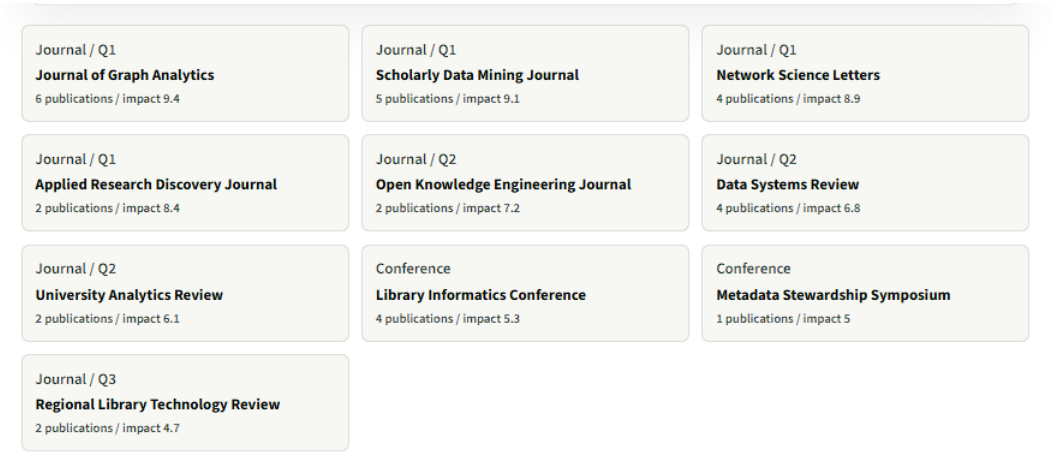


Figure 5: Analytics page showing the Q1 journal influence network and connected author relationships.

multi-graph membership, implements all three required queries, and includes graph visualization.

From a correctness point of view, the final project includes automated tests. The tests check seeded counts, graph membership,

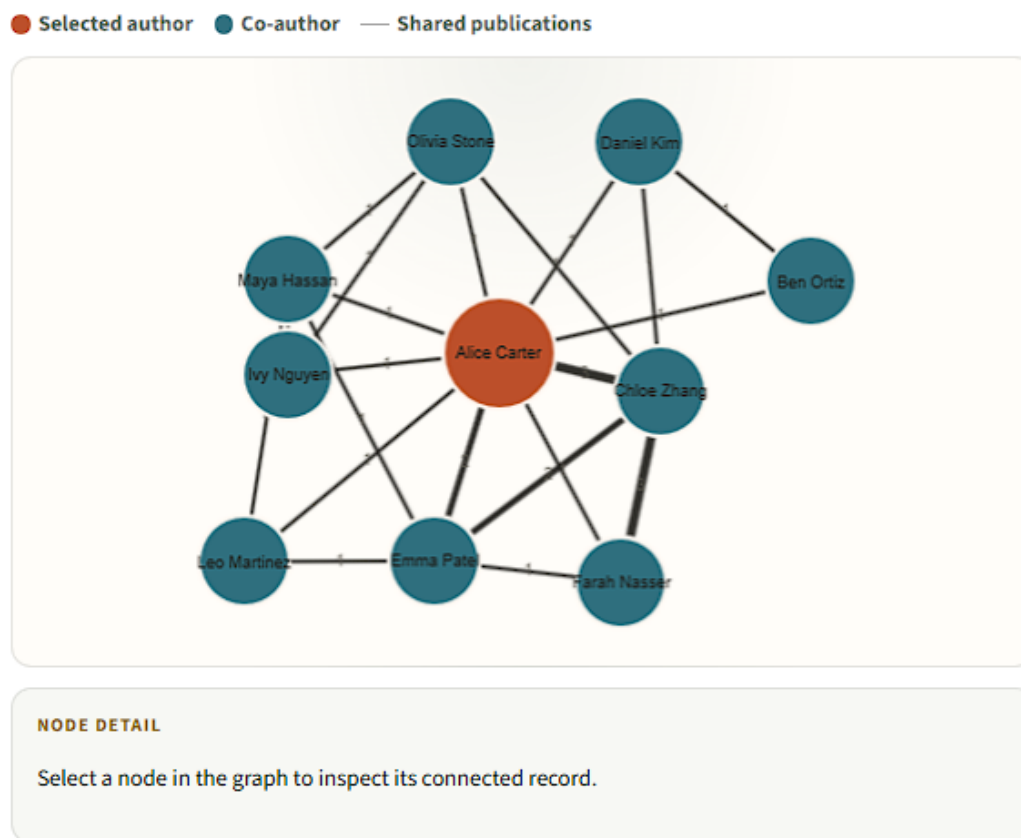


Figure 6: Graph visualization close-up showing interactive academic network exploration or export support.

required query behavior, h-index calculation, node attribute storage, filtered search, shortest-path behavior, influence ranking, and CRUD validation. Both the JavaScript graph service tests and the Python backend API tests passed in the final version.

From a usability point of view, the system is stronger than a minimal prototype. It includes a larger seeded dataset, readable dashboard, search filters, management forms, export options, and interactive graph exploration. These features make the project easier to demonstrate and easier for users to explore.

Table 1: Final Seeded Dataset Summary

Data Type	Count
Authors	18
Institutions	6
Venues	10
Publications	32
Named Graphs	3

13 Discussion

The main design decision in this project was to use a graph-oriented relational model instead of a pure graph database. The project still

uses SQLite tables, but the schema is organized around nodes and edges. This gave us a good balance between relational database structure and graph-style analysis.

One advantage of this design is simplicity. SQLite is easy to set up, easy to share, and works well for a class prototype. It also supports normal SQL queries, which made it easier to implement joins between authors, publications, venues, and citations.

Another advantage is flexibility. Since all entities are stored as nodes and all relationships are stored as edges, the same structure can support different graph views. The Collaboration Graph focuses on author relationships. The Citation Graph focuses on publication-to-publication citation links. The Venue Influence Graph focuses on authors, venues, and Q1 journal relationships.

However, this design also has tradeoffs. Some graph traversal operations are more complicated in SQL than they would be in a graph database like Neo4j. For example, deeper collaboration paths or community detection would require more complex queries or extra graph-processing code. For this project, SQLite was enough because the dataset is small and the required queries are clearly defined.

Overall, the graph-oriented relational approach worked well for the final prototype. It satisfied the project requirements and made the academic relationships easier to explain visually.

14 Challenges Faced

One challenge was designing the database so that it followed the project requirement while still being practical to query. The requirement asked for a shared Nodes table and a shared Edges table. At the same time, authors, institutions, publications, and venues all need different attributes. We solved this by storing shared identity fields in Nodes and storing type-specific fields in subtype tables.

Another challenge was computing h-index from graph data. The h-index query is not just a simple author lookup. It requires finding each author's publications, counting incoming citation edges for each publication, sorting the citation counts, and then applying the h-index rule. This required careful backend logic.

The Q1 influence query was also challenging because it combines multiple parts of the schema. The system has to find Q1 venues, find publications in those venues, find authors of those publications, and then find co-authors connected to those authors. This showed why graph modeling is useful, but it also required careful SQL and backend processing.

Frontend visualization was another challenge. Cytoscape.js requires nodes and edges to be formatted correctly. The frontend also needed different styles for selected authors, co-authors, Q1 publishers, and influenced authors. If a field was missing or mislabeled, the graph could display incorrectly. This required testing and debugging.

A final challenge was making the prototype feel complete. The required queries were the main priority, but the system also needed dashboard views, search, CRUD tools, export features, and a clean demo flow. These additions took extra time, but they made the final project stronger.

15 Limitations

Even though the final prototype is complete, it still has some limitations.

First, authentication is still demo-based. The system uses client-side local storage for login flow. This is useful for a class presentation, but it is not secure enough for a real production system.

Second, the system uses SQLite and a seeded dataset. SQLite is good for a lightweight class prototype, but it is not the best choice for very large academic datasets or heavy multi-user traffic.

Third, some analytics are simplified. The h-index query uses incoming citation edges from the seeded graph. The Q1 influence query uses venue quartile and co-author links. These are useful for the project requirements, but a real academic system would need more advanced analysis.

Fourth, the frontend has many useful features, but it does not yet include full browser-level automated testing. Most testing focuses on backend logic and API behavior.

16 Future Improvements

A future version should add real backend authentication. Passwords should be hashed, sessions should be managed on the server, and user permissions should be added.

The database could also be moved from SQLite to PostgreSQL or a specialized graph database such as Neo4j. This would make the system better for larger datasets and more complex graph traversal.

The dataset could be expanded by importing real academic records from sources like DBLP, OpenAlex, or ORCID. This would also require author disambiguation, because many researchers can have similar names.

More advanced analytics could also be added. Examples include PageRank-style influence, betweenness centrality, community detection, institution-level trends, and time-based graph animation.

The interface could be improved with saved query presets, better mobile support, stronger graph filtering, and printable report generation. Another future improvement would be a better reporting dashboard where users can automatically generate a summary of an author, institution, or venue network.

17 Conclusion

The final Academic Library Management System successfully delivers a complete graph-oriented academic library prototype. The system satisfies the main project requirements by storing all nodes in a shared Nodes table, storing all relationships in a shared Edges table, supporting multi-graph membership, implementing all required queries, and showing graph visualizations.

The final prototype also goes beyond the minimum requirements by adding CRUD tools, advanced search, larger seeded data, export features, node drill-down, additional analytics, and stronger validation. Overall, the project shows that a graph-based approach is effective for academic library data because it helps users understand collaboration, citation, and venue influence patterns in a visual and connected way.

Open Access and Code Availability Statement

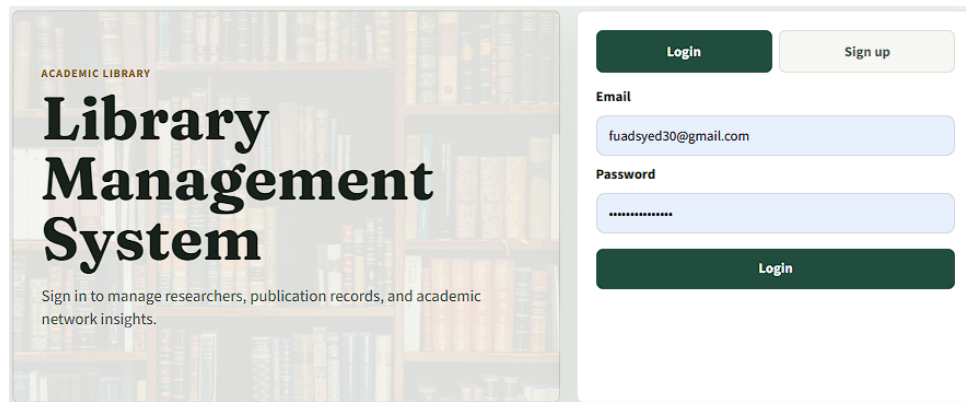
This report is open access under the MIT License. It can be shared without author consent for academic and research purposes, including use in future CS 3600 and CS 5602 classes.

The complete project code is shared on GitHub with a README file so others can run, study, and use the project:

<https://github.com/sthasafal/Academic-Library-Management-System>

A Additional Prototype Screenshots

The following screenshots show additional prototype features, including the login screen and record management interface.



ACADEMIC LIBRARY

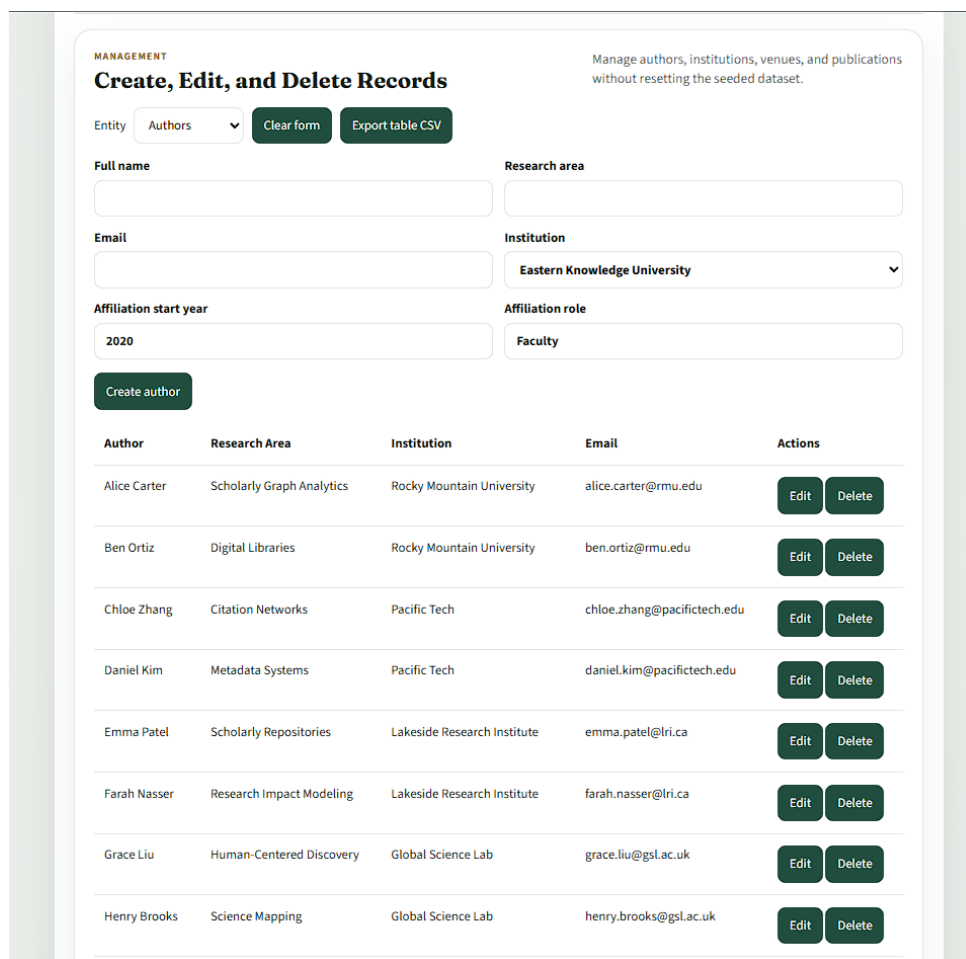
Library Management System

Sign in to manage researchers, publication records, and academic network insights.

Email

Password

Figure 7: Login and signup screen for the Academic Library Management System prototype.



MANAGEMENT

Create, Edit, and Delete Records

Manage authors, institutions, venues, and publications without resetting the seeded dataset.

Entity: Authors

Full name
Email
Affiliation start year
Research area
Institution Eastern Knowledge University
Affiliation role

Author	Research Area	Institution	Email	Actions
Alice Carter	Scholarly Graph Analytics	Rocky Mountain University	alice.carter@rmu.edu	Edit Delete
Ben Ortiz	Digital Libraries	Rocky Mountain University	ben.ortiz@rmu.edu	Edit Delete
Chloe Zhang	Citation Networks	Pacific Tech	chloe.zhang@pacifictech.edu	Edit Delete
Daniel Kim	Metadata Systems	Pacific Tech	daniel.kim@pacifictech.edu	Edit Delete
Emma Patel	Scholarly Repositories	Lakeside Research Institute	emma.patel@lri.ca	Edit Delete
Farah Nasser	Research Impact Modeling	Lakeside Research Institute	farah.nasser@lri.ca	Edit Delete
Grace Liu	Human-Centered Discovery	Global Science Lab	grace.liu@gsi.ac.uk	Edit Delete
Henry Brooks	Science Mapping	Global Science Lab	henry.brooks@gsi.ac.uk	Edit Delete

Figure 8: Management or CRUD screen showing create, edit, delete, or validation support for academic records.